

Knowledge Graphs and Quantum Computing: First Blood

Yannis Tzitzikas and Haridimos Kondylakis

¹ Computer Science Department, University of Crete, Heraklion, Greece

² Information Systems Laboratory, FORTH-ICS,

³ Center for Quantum Science & Technologies (FORTH-QuTech), GR-70013,
Heraklion (Crete), Greece tzitzik@csd.uoc.gr, kondylak@ics.forth.gr

Abstract. Although Quantum Computing (QC) is subject of intensive research, the data/knowledge management perspective is largely unexplored. Here we investigate how QC could be used for various tasks related to Knowledge Graphs (KG). We discuss the placement of QC in KG-related workflows, and then we (a) propose *encodings* of RDF (Resource Description Framework) data appropriate for QC, (b) show how a KG could be *stored* as a quantum state, (c) introduce a method for building *indexes* that is based on quantum circuits, (d) discuss how several important KG-related tasks, like *reasoning*, can be reduced to *satisfiability*, and thus gain from the quantum speedup. Subsequently, we report experimental results regarding encoding, decoding, superposition, as well as the limitations of the current QC hardware.

Keywords: quantum computing, knowledge graphs, knowledge management

1 Introduction

Quantum computing is subject of intensive research at various levels: from the *physical level* (i.e. how to build quantum computers and how to benchmark them [26]), to the *algorithmic level* (i.e., what algorithms are appropriate for quantum computers, e.g. the Shor’s algorithm [31] and the Grover’s algorithm [10], see [22] for an overview), up to *application level* (i.e. how to reduce various computational tasks to QC algorithms, e.g. see [6] for a survey of possible applications). In this paper we investigate how recent advances of QC could be leveraged for various tasks related to Knowledge Graphs (KG). The main motivation is that the properties of superimposed states could be exploited for storing a huge amount of RDF triples using a small number of qubits. Moreover, a mapping of triples to quantum states could enable native quantum algorithms to accelerate various KG tasks.

However, unlike classical computers, quantum computers cannot read data directly from the source. We need methods to *encode* our data to quantum states. Consequently, the general workflow is: (1) Pre-process the data, i.e. convert them to *numerical* data, (2) make these numerical data as the *input quantum*

state, (3) *run* the quantum computation, (4) *measure* the output of the quantum computation, (5) optional: use the output to update the input to the quantum state or the parameters of the quantum process, and (6) *decode* the result.

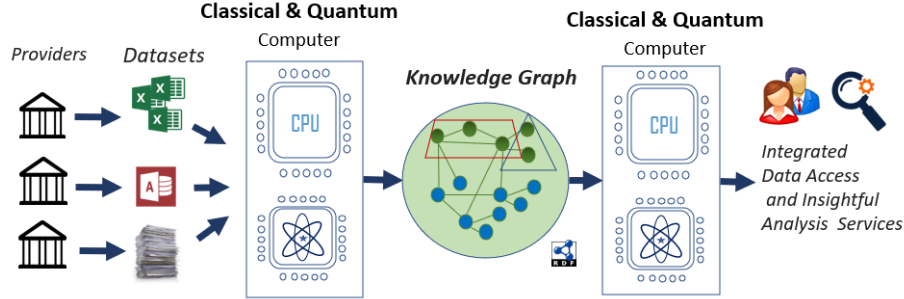


Fig. 1: The placement of QC in KG-related workflows

In general a quantum computer can be exploited both (I) for the construction of a KG, and (II) for the provision of KG-based services. This is illustrated in Figure 1. In the left side we foresee tasks like *entity matching*, *blank node matching*, *ontology learning*, and others. We could also think of *probabilistic/fuzzy KGs* represented in a quantum representation framework. In the right side we foresee tasks like *lookup*, *searching*, *querying*, *reasoning*, *similarity-based access services*, *question answering*, and others. However, in both "sides" we also need conventional computing, for handling the encoding and decoding, as illustrated in Figure 2.

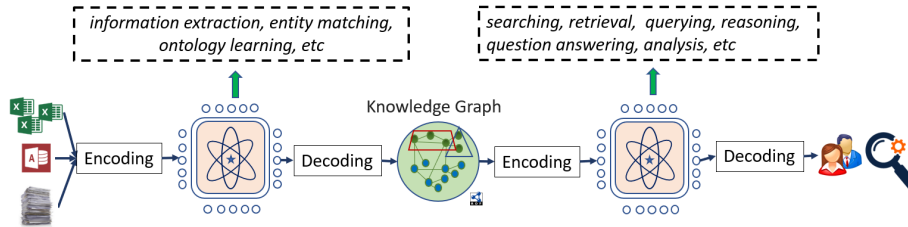


Fig. 2: Encoding and Decoding steps for various KG tasks

The main contributions of this paper are: we discuss the placement of QC in KG-related workflows, we introduce two basic *encodings* of RDF triples for QC, we detail a method for *quantum storage* for KGs, we show how quantum circuits can be used for building various *indexes*, we discuss how several important KG-related tasks, like *reasoning*, can be reduced to satisfiability (and thus gain from

the quantum speedup), and we report experimental results related to encoding, decoding the superposition.

2 Background

2.1 Quantum Computing

In QC (Quantum Computing) instead of representing the data with 0/1 bits, we represent data with *qubits*. A qubit is described by a state vector in a two-dimensional complex vector Hilbert space. We express a qubit as an expression $a|0\rangle + b|1\rangle = a \begin{bmatrix} 1 \\ 0 \end{bmatrix} + b \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} a \\ b \end{bmatrix}$ where $a^2 + b^2 = 1$. Unlike probabilities, where $a, b \in [0, 1]$ and $a + b = 1$, here a and b can be complex numbers, however a^2 and b^2 have a probabilistic interpretation ($a^2, b^2 \in [0, 1]$, $a^2 + b^2 = 1$). A qubit $a|0\rangle + b|1\rangle$ where $a, b \neq 0$, is *simultaneously* in two states, this is called *superposition*, and only when we measure it, we will get either 0 or 1. As such a system with n qubits can represent 2^n states at once. The "magic" is that what we will measure (0 or 1) may depend on the states of other qubits, due to the phenomenon of interference. We could therefore say that we have a kind of entangled probabilities. In particular, if we have two *entangled* qubits (that in the physical word correspond to two entangled electrons/photons), they might appear to be independent particles in a superposition, but as soon as one of them is measured, the outcome of a measurement of the other is fixed as well. Many computational benefits of QC stems from superposition, i.e. that fact that a quantum system can do processing while the qubits are in a superposition state. Suppose, for example, that we have 10 qubits. The operations defined by a quantum algorithm do not manipulate only 10 bits; they manipulate $2^{10} = 1024$ probability values. One step in a quantum algorithm on a quantum computer with 10 qubits is modifying 1024 values. Adding one qubit doubles the processing capabilities of the quantum computer. This explains the term exponential, which is often used in QC: adding n qubits adds processing power proportional to 2^n .

A bit formally, qubits can be combined to form *quantum registers*. The joint space of n qubits is the tensor product of the individual qubit spaces, i.e. $H = H_1 \otimes \dots \otimes H_n$, e.g. two qubits $|\psi_1\rangle = a_1|0\rangle + b_1|1\rangle$ and $|\psi_2\rangle = a_2|0\rangle + b_2|1\rangle$ form the joint space $|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle = a_1a_2|00\rangle + a_1b_2|01\rangle + b_1a_2|10\rangle + b_1b_2|11\rangle$. The state of a quantum register Ψ is described by a unitary⁴ transformation (U) that acts on the full state space, i.e. $|\Psi'\rangle = U|\Psi\rangle$.

There are not many quantum algorithms, for an overview of quantum algorithms see [22] and [16]. The more important ones are: the Shor's algorithm [31] that solves the factorization problem in polynomial time, and the Grover's algorithm [10] for searching in an unstructured list at the cost of $\sqrt{n}$. We should also mention quantum annealing [29] that relies on specialized hardware (D-Wave) for solving optimization problems.

⁴ Unitary transformations govern time evolution (via the Schrödinger equation), ensuring probabilities are conserved.

2.2 Knowledge Graphs

Knowledge Graphs (KG) [12] are structured representations of data so that they allow capturing the relationships between different concepts, entities or pieces of information. They rely on a graph-based data model to capture knowledge in application scenarios that involve integrating, managing and extracting value from diverse data sources [24]. There are several KGs expressed in RDF that integrate data from various sources: from general purpose sources like DBpedia [4] and Wikidata [34], to domain-specific repositories (e.g., Europeana [14], Drug-Bank [36], GRSF [32], ORKG [15], WarSampo [17] other cultural datasets like [9], COVID-19 related datasets [8], as well as Markup data through `schema.org`. Recently KGs, are also used for improving AI-related services, e.g. the validation of answers from Large Language Models [23], as well as in RAG systems [30].

Based on their construction method, we could separate KGs in three distinct cases: (a) KG that are constructed using information extraction from free text (e.g. DBpedia⁵, OpenIE⁶, Wikidata⁷, etc.), (b) KGs that are generated by transforming structured data using various methods and tools (e.g. Europeana [14], Microsoft Academic Graph [35], etc.), and KGs that are constructed by combining transformed resources from structured data and information extracted from free text (e.g. [28]).

KGs in RDF. Currently most knowledge graph data management systems use Resource Description Framework (RDF) or Property Graph (PG) models. In this paper we focus on RDF. RDF uses *triples* i.e. statements of the form *subject – predicate – object*, where the *subject* corresponds to an entity (e.g. a product, a company etc.), the *predicate* to a characteristic of the entity (e.g. price of a product, location of a company) and the *object* to the value of the predicate for the specific subject (e.g. “300”, “US”). The triples are used for relating Uniform Resource Identifiers (URIs) or anonymous resources (blank nodes) with other URIs, blank nodes or constants (Literals). Formally, a *triple* is considered to be any element of $T = (U \cup B) \times (U) \times (U \cup B \cup L)$, where U , B and L denote the sets of URIs, blank nodes and literals, respectively. Any finite subset of T constitute an *RDF graph* (or *RDF data set*).

3 QC and Data/Knowledge Management

Works on QC and *data management* have started to appear. For instance, [11] is a recent tutorial that gives an overview of works on QC-based data management (query optimization, data integration, transaction mgmt, etc). [18] focuses on *quantum storage design* for Tables in RDBMS. They propose two storage methods, Quantum Column-oriented Store and Quantum Row-oriented Store, tailored to store relational tables on universal quantum computers.

⁵ <https://www.dbpedia.org/>

⁶ <https://stanfordnlp.github.io/CoreNLP/openie.html>

⁷ https://www.wikidata.org/wiki/Wikidata:Main_Page

There are a few works on Knowledge Graphs too: [3] focuses on *entity matching* with *Quantum Neural Networks*; they evaluate quantum machine learning algorithms for entity matching on a small handcrafted data set and compare them to similar classical algorithms. [20] proposes a variational quantum circuit model for *knowledge graph embeddings*. The latent representations of entities are encoded as coefficients of quantum states, while predicates are characterized by parametric gates acting on the quantum states. They evaluate the result through link prediction experiments. Finally, [19] proposes a quantum *machine learning* algorithm for knowledge graphs.

A general remark is that in all of these works, the *data management* perspective is largely unexplored and quite far from technological and practical exploitation.

4 KG Encoding for Quantum State Input

We shall use a small running example. Consider the following three RDF triples:

```
(ex:Aristotle, rdf:type, ex:Person)
(ex:Person, rdfs:subClassOf, ex:Mortal)
(ex:Aristotle, ex:livesAt, ex:Athens)
```

We have to encode them *numerically*. Figure 3 shows two possible encodings for the URIs, that we shall call *R1* and *R2*. According to *R1* we assign to each URI a number, while according to *R2* we assign *independently* numbers to URIs corresponding to entities and to those corresponding to predicates (this is sound since a predicate cannot appear as subject or object of a triple).

<i>R1</i> encoding	<i>R2</i> Encoding
<pre>0(000): rdf:type 1(001): rdfs:subClassOf 2(010): ex:Aristotle 3(011): ex:Person 4(100): ex:Mortal 5(101): ex:livesAt 6(110): ex:Athens</pre>	<pre>0(00):rdf:type 1(01):rdfs:subClassOf 2(10):ex:livesAt 0(00):ex:Aristotle, 1(01):ex:Person 2(10):ex:Mortal 3(11):ex:Athens</pre>

Fig. 3: Two encodings for RDF triples

According to *R1* the triple `(ex:Aristotle, rdf:type, ex:Person)` is mapped to `(2,0,3)`. In binary, and assuming 3 bits, it is mapped to `(010,000,011)`, therefore this triple is mapped to the quantum state $|010, 000, 011\rangle$, so we need 9 qubits. According to *R2*, the same triple is mapped to `(0,0,1)`, in binary to `(00,00,01)`, i.e. to the quantum state $|00, 00, 01\rangle$, so here we need 6 qubits. The fact that in

R2 the two bits 00 corresponds to `rdf:type` and `ex:Aristotle`, does not create any ambiguity, since a predicate cannot appear as subject or object of a triple, and we used fixed number of bits.

Comparison of $R1$ and $R2$. Let E be the set of entities, and P the set of predicates of a KG. With the $R1$ encoding each triple needs $3 \lceil \log_2(|E| + |P|) \rceil$ bits. According to the $R2$ encoding each triple needs $2 \lceil \log_2(|E|) \rceil + \lceil \log_2(|P|) \rceil$ bits. In a particular KG, it is therefore beneficial to select the representation that gives the less number of bits. An illustration of the number of qbits required by each one is shown in Figure 4. Roughly, $R1$ is better than $R2$, if $3 \log_2(|E| + |P|) < 2 \log_2(|E|) \log_2(|P|) \Leftrightarrow (|E| + |P|)^3 < |E|^2 * |P|$. However this cannot hold, so in general $R2$ is beneficial (apart from some cases due to rounding).

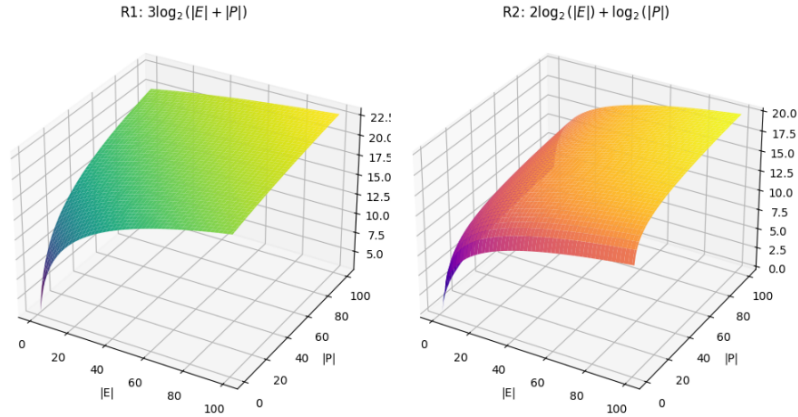


Fig. 4: # qubits required by $R1$ and $R2$, for various values of $|E|$ and $|P|$

4.1 Case Study: DBpedia

As of the latest available data⁸, DBpedia contains over 850 million RDF triples, representing structured information extracted from Wikipedia and other sources. The dataset includes approximately 55,000 properties, with 1,377 defined by the DBpedia ontology. DBpedia encompasses around 7.62 million entities linked to 179 external sources, totaling over 130.6 million links. The dataset is multilingual, featuring labels and abstracts in 125 languages, and includes approximately 38 million labels, 25.2 million image links, and 29.8 million links to external web pages. That is:

$|T| = 850$ millions, i.e. $\lceil \log(|T|) \rceil = 30$,

⁸ <https://www.dbpedia.org/blog/dbpedia-snapshot-2022-03-release>

$|E| = 7.62$ millions, i.e. $\lceil \log(|E|) \rceil = 23$,
 $|P| = 55.000$, i.e. $\lceil \log(|P|) \rceil = 16$.

Suppose we want to apply a QC task on the entire DBpedia. Each DBpedia triple will require:

- $R1$: $3 \lceil \log_2(|E| + |P|) \rceil$ bits. $= 3 * (23) = 69$ bits,
- $R2$: $2 \lceil \log_2(|E|) \rceil + \lceil \log_2(|P|) \rceil = 2 * 23 + 16 = 62$ bits.

Note that this number of qubits (62) is not that big as already IBM offers through its cloud infrastructure 126 qubits to the public.

5 Quantum Storage for KGs

Motivation. Through the properties of superposition states, one could store a huge amount of triples in a small number of qubits. Moreover, a mapping of triples to quantum states could enable native quantum algorithms to accelerate various KG tasks.

Method. However, as mentioned earlier, we have to encode everything with numbers, therefore a kind of *lexicon*, like the one shown in Figure 3 has to be kept in a conventional computer. Therefore we need to have stored: (a) the actual Triples, (b) the URI Encoding Lexicon, (c) the EncodedTriples. If n is the number of triples, then the encoding costs $O(n \log n)$ time. If the result of a QC task is a triple, then its decoding costs $O(\log n)$ time. Note that such (bitmap) indexes are already used by triple stores to minimize storage cost and operation size.

Assume that we have this lexicon, and we have decided the encoding to use. Let us now focus on the quantum storage. Let T be the set of triples of our KG. For a $t \in T$, we shall use $\psi(t)$ to denote its encoding (either $R1$ or $R2$). We can store these by *superimposing* them, i.e. we can store the state $\Psi_T = \frac{1}{\sqrt{|T|}} \sum_{t \in T} \psi(t)$. In our running (and assuming $R2$), this means that we store in a quantum computer the state $\frac{1}{\sqrt{3}}(|000001\rangle + |010110\rangle + |001011\rangle)$ which equals a 64-dimensional state vector ($2^6 = 64$), with all entries zero except at positions 1, 11, and 22 where the amplitude is $\frac{1}{\sqrt{3}}$.

5.1 Quantum Lookup

Suppose that we have n RDF triples, i.e. $|T| = n$. If we have not created any index, then to perform *lookup*, i.e. to check if there is one particular triple t , then we have to scan $n/2$ elements in average. Now suppose the quantum representation described in §5. In a quantum computer, and with the Grover's algorithm [10] this can be decided at $\sqrt{n}$ steps. Returning to our running example, suppose that we want to lookup the triple `(ex:Aristotle, rdf:type, ex:Person)`, and assume that we have adopted the $R2$ encoding, i.e. we want to lookup 000001, i.e. 6 bits. The Qiskit implementation of the Grover's algorithm returns the right triple with probability 0.996.

6 Indexing through Quantum Circuits

Suppose that apart from plain lookup we would like to construct *indices* for speeding up various operations, i.e. those required by a query language like SPARQL, for evaluating various triple patterns. We can try constructing *dedicated quantum circuits* for this purpose. To capture the basic access requirements, we would like to have 6 functions, i.e. one function f_a for each $a \in A = \{S \rightarrow P, S \rightarrow O, P \rightarrow S, P \rightarrow O, O \rightarrow S, O \rightarrow P\}$. For instance, $f_{S \rightarrow P}$ aims at speeding up the task of finding the predicates that are associated to a given subject, and it is defined as $f_{S \rightarrow P}(s) = \{p | (s, p, o) \in T\}$. The rest functions are defined analogously. Note that we could employ extra functions to speed up more access patterns, i.e. the following three: $A^+ = \{S \rightarrow PO, P \rightarrow SO, O \rightarrow PS\}$, e.g. $f_{S \rightarrow PO}(s) = \{(p, o) | (s, p, o) \in T\}$.

Let us now see how we can implement each such function using a quantum circuit. To aid understanding, we will present the idea through our running example. Let's consider the function $f_{P \rightarrow O}$ that aims at finding fast all *objects* that are associated with a particular predicate. In Table 1 we can see the three triples of our running example and their encoding (according to $R1$), for convenience in decimal. For the needs of $f_{P \rightarrow O}$ we have to consider only the columns for p and o , Table 2 shows these columns, and their contents in binary, as well as their mappings to 6 bit positions named $a_0, \dots, b_2$.

s	$\psi(s)$	p	$\psi(p)$	o	$\psi(o)$
ex:Aristotle	2	rdf:type	0	ex:Person	3
ex:Person	3	rdfs:subClassOf	1	ex:Mortal	4
ex:Aristotle	2	ex:livesAt	5	ex:Athens	6

Table 1: Running example with $R1$ encoding

$\psi(p)$	$\psi(o)$	$\psi(p)$	$\psi(o)$	a_0	a_1	a_2	b_0	b_1	b_2
0	3	000	011	0	0	0	0	1	1
1	4	001	100	0	0	1	1	0	0
5	6	101	110	1	0	1	1	1	0

Table 2: Data related to $f_{P \rightarrow O}$

We can now express each of b_0, b_1, b_2 , as a boolean function of a_0, a_1, a_2 . In our case we get $b_0 = \overline{a_1} \wedge a_2$; $b_1 = (\overline{a_0} \wedge \overline{a_1} \wedge \overline{a_2}) \vee (a_0 \wedge \overline{a_1} \wedge a_2)$; $b_2 = \overline{a_0} \wedge \overline{a_1} \wedge \overline{a_2}$. We can turn these boolean formulas into quantum circuits, based on the work [37] that provides an efficient technique that can implement any given Boolean

function as a quantum circuit. In general, a boolean function with $n - 1$ inputs, is represented by a quantum circuit with n qubits, where the extra qubit is initialized with 0 and will carry the result of the Boolean function at the end of the computation. Figure 5 shows the resulting circuit. It comprises X (Pauli-X) gates and CNOT (Controlled Not) gates. The X gate is the quantum equivalent of the classical NOT gate: it flips a qubit: it turns $|0\rangle$ into $|1\rangle$ and $|1\rangle$ into $|0\rangle$. The CNOT gate is a multi-qubit gate. It flips (applies a NOT operation to) the target qubit (the one marked with a plus) if the control qubit(s) is/are $|1\rangle$. We have annotated the circuit (of Figure 5) with the logical expression after each gate. To conclude, we have constructed a circuit that whenever we feed it with the bits of a predicate we get as output the bits of the object.

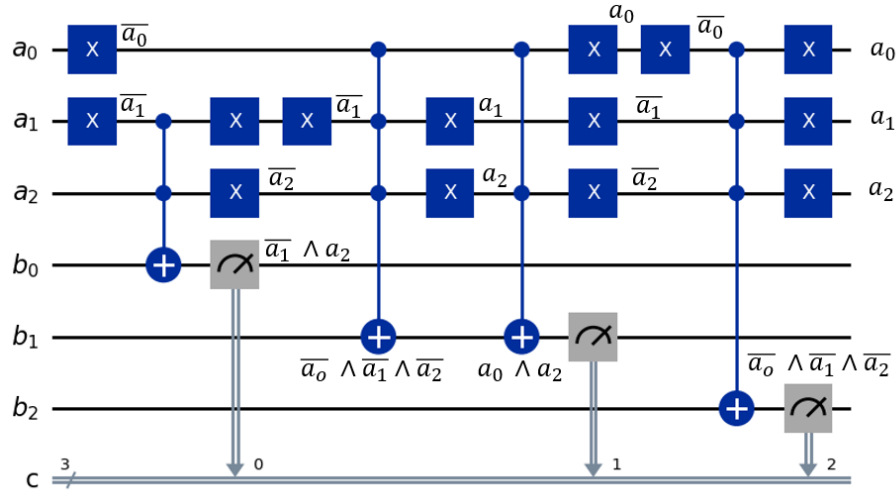


Fig. 5: The resulting quantum circuit (the boolean formulas after each gate were added manually for explainability)

However, this is possible only if f is a proper function. If p is a *functional property* (or effectively functional), then we can use $f_{SP \rightarrow O}$. If p is an *inverse functional property*, then we can use $f_{OP \rightarrow S}$. If pp denotes a property path, and pp is functional, then $f_{SP \rightarrow P}$ can be used as well. Now suppose that we want to apply this idea (i.e. build a quantum circuit for indexing purposes), on a binary relation f that is not a function, e.g. in our running example $f_{S \rightarrow O}(\text{Aristotle}) = \{ex : \text{Person}, ex : \text{Athens}\}$. In such cases we can create a *lexicon of sets*, where each set is assigned a distinct identifier, and use these identifiers in our quantum index (turning in this way f to a proper function). Let 7 be the identifier assigned to the set $\{ex : \text{Person}, ex : \text{Athens}\}$. In this case $f_{S \rightarrow O}(\text{Aristotle}) = 7$. If the set identifiers, are integer values greater than the maximum integer value assigned to URIs (either in $R1$ or $R2$), then this prevents ambiguities. This *lexicon of*

sets, can be stored in a conventional data structure, a compact one is the CPOI (Compact Partial Index) [33, 27].

To synopsise, what we have seen is that apart from using Grover (and having lookup cost $\sqrt{n}$) we can construct and use an index to speedup access. The cost of using the index is based on the number of gates it has. *Gate requirements*: As regards the number of gates required, according to [37], the maximum number of CNOT gates for representing a function over n qubits is 2^{n-1} and the average is 2^{n-2} . This means that for building a quantum circuit for representing a function over 62 qubits (recall the requirements of DBpedia) we need 2^{60} CNOT gates in the average case (more in §8).

7 Satisfiability, Reasoning and Instance Checking

Satisfiability is a key reasoning task. It is the problem of checking whether a knowledge base K is *satisfiable*, i.e., whether it has a model. Below we shall use notation from Description Logics (DL). It is well known that many reasoning tasks, i.e. *concept satisfiability*, *subsumption checking*, *instance checking*, and others, reduce to *satisfiability*. A concept C is satisfiable, i.e. $K \not\models C \equiv \perp$, iff there exists x such that $K \cup \{C(x)\}$ is satisfiable. *Concept subsumption*, i.e. $K \models C \sqsubseteq D$, is reducible to satisfiability: $K \models C \sqsubseteq D$ iff there exist x such $K \cup \{C \sqcap \neg D\}(x)$ is not satisfiable. *Instance checking*, i.e. $K \models C(a)$, is again reducible to satisfiability: i.e. $K \models C(a)$ iff $K \cup \{\neg C(a)\}$ is not satisfiable.

For instance, our running example is expressed in DL as:

$K = \{\text{Person}(\text{Aristotle}), \text{Person} \sqsubseteq \text{Mortal}, \text{livesAt}(\text{Aristotle}, \text{Athens})\}$

To check whether $(\text{ex:Aristotle}, \text{rdf:type}, \text{ex:Mortal})$ holds, which is an *inferred triple* according to the semantics of RDF Schema, is equivalent to checking whether $K \models \text{Mortal}(\text{Aristotle})$, which holds iff $K \cup \{\neg \text{Mortal}(\text{Aristotle})\}$ is not satisfiable.

The complexity of satisfiability in OWL is described in [13, 25], in brief, the complexity of satisfiability in OWL Lite is EXPTIME-complete, in OWL DL it is NEXPTIME-complete, while in OWL 2 DL it is N2ExpTime-complete.

QC and Satisfiability. Note that Grover’s algorithm can be applied for solving the satisfiability problem in a quantum computer in polynomial time! (related publications include [2] and [5]). It follows that satisfiability (and the tasks that reduce to satisfiability, like instance checking, and concept subsumption) could be solved better on a quantum computer than on a conventional computer.

Of course this is not the only method to perform such tasks. For instance, suppose that we have two edges (a,b) and (b,c) and we want to compute the *transitive closure*, i.e. $\{(a,b), (b,c), (a,c)\}$. We can represent this binary relation as a quantum circuit where each qubit corresponds to an edge. By applying controlled operations, we can infer new edges based on transitivity. The circuit is shown in Figure 6. We use two X gates to set 1 the qubits corresponding to (a,b) and (b,c) . Then we use a CCX gate (Toffoli gate) that takes three qubits as input. If the first two qubits are both 1, it flips the third qubit. This is just

indicative, other solutions, based on *quantum walks* are worth investigating as well.

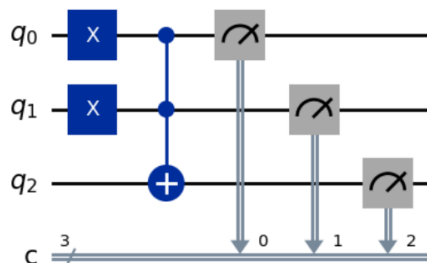


Fig. 6: Circuit for computing the transitive closure of a relation with two edges

In brief, the reduction to satisfiability can be used to carry out various tasks and leverage the polynomial complexity that QC offers, without restricting other algorithms from dedicated tasks.

8 Experimental Validation and Limitations

A thorough experimental evaluation is beyond the scope of this paper. However, we performed various validation tests, and some experiments for some indispensable parts of the quantum computing pipeline for RDF data.

Validation. At first, all examples and methods of this paper have been tested over IBM Qiskit 2.0.0.

Encoding and Decoding. We have developed **QuantumEncoDeco**, a framework for *R1* and *R2* encoding/decoding of RDF data, and we have made it publicly available⁹, enabling anyone to encode/decode RDF graphs, and make QC experiments. The encoding creates also the lexicon that is useful for decoding the results of a QC task. Since encoding and decoding are required for any task, we have to anticipate these costs, therefore in Table 3 we report experimental results¹⁰ over few datasets of different sizes. The datasets include (a) the ISO Standard ontology CIDOC-CRM v.7¹¹, a small part of DBpedia¹², GRSF Aquatic Thesaurus [21] that contains around 1.6 million triples with marine species, and the ChEBI (Chemical Entities of Biological Interest) ontology¹³, a freely available dictionary of molecular entities focused on small chemical compounds, that

⁹ <https://github.com/YannisTzitzikas/QuantumEncoDeco>

¹⁰ All measurements were performed using a machine with Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz 2.11 GHz, 16.0 GB RAM.

¹¹ <https://cidoc-crm.org/>

¹² In particular <https://downloads.dbpedia.org/2016-10/dbpedia\2016-10.owl>

¹³ <https://www.ebi.ac.uk/chebi/>

contains around 5 million triples. We observe that $R2$ is significantly better than $R1$ in all cases. We can see that the encoding of datasets that have tens of thousands of triples, takes a few seconds, while the encoding of 5.2 million triples takes around 2 minutes, and the full decoding 0.5 minutes.

Dataset	$ T $	$ E $	$ P $	Encoding	Bits per triple	enco. time	deco. time
CIDOC_CRM_v7.1.1	3,886	2,582	9	$R2$	28	1.8 s	0.2 s
				$R1$	36	1.2 s	0.4 s
DBpedia small	31,050	20,214	23	$R2$	35	0.72 s	0.3 s
				$R1$	45	1.8 s	0.4 s
GRSF	1.6 million	0.58 million	8	$R2$	44	43.8 s	4 s
				$R1$	40	24.6 s	10 s
ChEBI	5.2 million	1.49 million	39	$R2$	48	1.9 min	0.49 min
				$R1$	63	1.6 min	0.6 min

Table 3: Results regarding encoding and decoding using $R1$ and $R2$

Superposition time. After encoding a KG, the next step is to express it as a quantum state. Here we report times required for *creating the superimposed state*, assuming that the encoded data are stored in a file. We measure the time required for reading it, for loading it in main memory, for creating the state vector, for creating the quantum circuit from the state vector, and for the transpilation (the process of transforming a quantum circuit into an equivalent one that is optimized and compatible with a specific quantum hardware, in our case IBM Quantum Computer). The implementation was done in Qiskit, the results are shown in Table 4. We observe that for 1 million bitwords each with 20 bits, the construction of the superposition takes in total 3 seconds, while for 33 million bitwords each with 25 bits, the construction of the superposition takes in total 133 seconds, i.e. around 2 minutes.

Data set	#Qubits	#Bitstrings	Loading time (from file to main mem)	State vector creation	Create circuit from state vector	Transpile	Total time	Simulator run
s1	12	1,000	1 ms	2 ms	14 ms	210 ms	213 ms	6 ms
s2	15	32,000	1 ms	6 ms	94 ms	142 ms	243 ms	6 ms
s3	20	1,000,000	327 ms	391 ms	1.46 s	462 ms	2.6 s	341 ms
s4	25	33,554,432	12 s	11.7 s	50.3 s	31.1 s	133 s	27.9 s

Table 4: Times required for creating the superposition

Hardware/Simulator Limitations. However we should note that it is rather infeasible (for the time being) to perform large scale experiments. If one uses a *simulation* (like Qiskit Aer), then the number of supported qubits is rather small, since the simulation of qubits on a classical computer requires exponential memory, therefore Qiskit Aer can support up to 25 qubits. Even the number of logical qubits of the IBM Quantum computer that one can use, is limited, i.e. 133. In addition there is the limit of 10 minutes free per month.

Number of Gates Limitation. The IBM Quantum computer has various other limits¹⁴ as regards the *number of gates* (the maximum number of two-qubit gates per circuit is 5 million for static circuits and 100,000 for dynamic circuits), as well as the *instructions per qubit* (the service permits up to 26.8 million control-system instructions per qubit). Recall (from §6) that the number of gates required for making a quantum index over DBpedia is 2^{60} CNOT gates, far beyond what it is currently available (i.e. IBM supports 2^{22}). We hope that soon more capabilities and less restrictions will be available.

Grover’s algorithm. Grover’s algorithm is of prominent importance for lookup over unstructured data (recall §5), and for satisfiability checking (recall the KG tasks that are reducible to satisfiability §7). However, the application of this algorithm on real Quantum Computers with *many qubits* is largely unexplored: no results of effective application of this algorithm on more than 10 qubits have been reported. Even the behaviour over 3 qubits is subject of current research, e.g. [7, 1] investigate the performance and error rate of Grover’s algorithm over 3 qubits. Consequently, one can only use a simulator but this has the limitation of 25 qubits, and the run is slow (since a classical computer simulates explicitly the exponential in number cases that are stemmed by superposition). Below we report a few indicative experimental results using the Aer simulator. At first note that the number of iterations k that is required by Grover’s algorithm depends on the search space, i.e. the number of qubits n , and the number of marked items M , in particular $k \approx \frac{\pi}{4} \sqrt{\frac{2^n}{M}}$, e.g. for $M=1$ we get $k \approx \frac{\pi}{4} \sqrt{2^n}$. E.g. for 20 qubits we need approx. 840 for $M=1$ and 568 for $M=2$. For this particular case, the execution time of the Grover operator takes 0.29 seconds, the transpilation takes 3.2 seconds and execution of 10 shots costs 48 seconds. However, all these are simulator results. As stated, the application of Grover on real quantum hardware with this number of qubits is not currently possible.

9 Concluding Remarks

Quantum computing and KGs is an unexplored area. In this paper we provided a starting point with emphasis on the data management perspective. We are interested in data management from start to finish, in exploring the current limitations, and in the verifiability of each step. We discussed the possible placement of QC in KG-related workflows, and then we showcased two basic *encodings* and how to estimate the number of qubits that are required for superimposing a KG (on the basis of $|E|$ and $|P|$). Then we showed how we can *store* a KG as a quantum state (e.g. the entire DBpedia can be superimposed on 62 qubits). To enable fast access, we showed how quantum circuits can be used for building various *indexes* and estimated the number of quantum gates that are required. Since the problem of satisfiability can be solved in polynomial time in a quantum

¹⁴ <https://docs.quantum.ibm.com/guides/job-limits>

computers, we showed how several important KG-related tasks, like *reasoning*, can be reduced to satisfiability, and thus gain from the quantum speedup.

In addition, we developed and made publicly available **QuantumEncoDeco**, a library for encoding and decoding RDF data, and we reported experimental results regarding the encoding, decoding and superposition of RDF triples. Indicatively, a dataset with around 5 million RDF triples requires around 2 minutes to encode, but only a few seconds to superimpose these encoded triples.

As future research we plan to investigate how to apply the Grover’s algorithm, and get stable results, for *high* number of qubits. Another topic for future research is to investigate the potential of QC in combinatorial (apart from satisfiability) and optimization problems (like entity matching, blank node matching, SPARQL optimization, etc.). Such problems seem suitable for *quantum annealing*; in this paper we focused on the *quantum circuit model*.

Acknowledgments. This work was supported by DEDALUS - Data Management using Quantum Computing project (TA 5180519) funded by SUB1.1 Clusters of Research Excellence (CREs), Greece. We also thank Giorgos Theodorakis for contributing in the code of **QuantumEncoDeco**, as well as George T. Stamatiou for comments and feedback on earlier versions of this paper.

References

1. AbuGhanem, M.: Characterizing grover search algorithm on large-scale superconducting quantum computers. *Scientific Reports* **15**(1), 1281 (2025)
2. Battaglia, D.A., Santoro, G.E., Tosatti, E.: Optimization by quantum annealing: Lessons from hard satisfiability problems. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* **71**(6), 066707 (2005)
3. Bischof, L., Teodoropol, S., Füchslin, R.M., Stockinger, K.: Entity matching with quantum neural networks (2024)
4. Bizer, C., Lehmann, J., Kobilarov, G., Auer, S., Becker, C., Cyganiak, R., Hellmann, S.: DBpedia-A crystallization point for the web of data. *Journal of web semantics* **7**(3), 154–165 (2009)
5. Boulebnane, S., Montanaro, A.: Solving boolean satisfiability problems with the quantum approximate optimization algorithm. *PRX Quantum* **5**(3), 030348 (2024)
6. Dalzell, A.M., McArdle, S., Berta, M., Bienias, P., Chen, C.F., Gilyén, A., Hann, C.T., Kastoryano, M.J., Khabiboulline, E.T., Kubica, A., et al.: Quantum algorithms: A survey of applications and end-to-end complexities. *arXiv preprint arXiv:2310.03011* (2023)
7. Daoud, A., Millberg, V.: Performance and noise levels of grover’s algorithm on quantum computers with different qubit connectivities. *Tech. rep.*, KTH, School of Electrical Engineering and Computer Science (2024)
8. Dimitrov, D., Baran, E., Fafalios, P., Yu, R., Zhu, X., Zloch, M., Dietze, S.: Tweetscov19—a knowledge base of semantically annotated tweets about the covid-19 pandemic. In: *29th ACM International Conference on Information and Knowledge Management (CIKM 2020)* (2020)
9. Fafalios, P., Petrakis, K., Samaritakis, G., Doerr, K., Kritsotaki, A., Tzitzikas, Y., Doerr, M.: FAST CAT: Collaborative data entry and curation for semantic

- interoperability in digital humanities. *ACM Journal on Computing and Cultural Heritage* **14** (2021)
10. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219 (1996)
11. Hai, R., Hung, S.H., Feld, S.: Quantum data management: From theory to opportunities. *arXiv preprint arXiv:2403.02856* (2024)
12. Hogan, A., Blomqvist, E., Cochez, M., d’Amato, C., Melo, G.D., Gutierrez, C., Kirrane, S., Gayo, J.E.L., Navigli, R., Neumaier, S., et al.: Knowledge graphs. *ACM Computing Surveys (Csur)* **54**(4), 1–37 (2021)
13. Horrocks, I., Patel-Schneider, P.: Reducing OWL entailment to description logic satisfiability. *Journal of Web Semantics* **1**(4), 345–357 (2004)
14. Isaac, A., Haslhofer, B.: Europeana linked open data–data. *Europeana. eu. Semantic Web* **4** (2013)
15. Jaradeh, M.Y., Oelen, A., Farfar, K.E., Prinz, M., D’Souza, J., Kismihók, G., Stocker, M., Auer, S.: Open research knowledge graph: Next generation infrastructure for semantic scholarly knowledge. In: *Proceedings of the 10th International Conference on Knowledge Capture*, pp. 243–246 (2019)
16. Jordan, S.P.: Quantum algorithm zoo. <https://quantumalgorithmzoo.org>
17. Koho, M., Ikkala, E., Leskinen, P., Tamper, M., Tuominen, J., Hyvönen, E.: Warsampo knowledge graph: Finland in the second world war as linked open data. *Semantic Web – Interoperability, Usability, Applicability* (2020). <https://doi.org/10.3233/SW-200392>
18. Li, T., Yuan, G., Yao, C., Shi, M., Wang, Z., Qian, L., Lu, J.: Quantum storage design for tables in RDBMS. *Proceedings of the VLDB Endowment*. ISSN **2150**, 8097 (2024)
19. Ma, Y., Tresp, V.: Quantum machine learning algorithm for knowledge graphs. *ACM Transactions on Quantum Computing* **2**(3), 1–28 (2021)
20. Ma, Y., Tresp, V., Zhao, L., Wang, Y.: Variational quantum circuit model for knowledge graph embedding. *Advanced Quantum Technologies* **2**(7-8), 1800078 (2019)
21. Marketakis, Y., Tzitzikas, Y., Gentile, A., Ellenbroek, A., Taconet, M.: Building a global aquatic resource knowledge base for fisheries. *Proceedings* **117**(1) (2025). <https://doi.org/10.3390/proceedings2025117004>
22. Montanaro, A.: Quantum algorithms: an overview. *npj Quantum Information* **2**(1), 1–8 (2016)
23. Mountantonakis, M., Tzitzikas, Y.: Real-time validation of ChatGPT facts using RDF knowledge graphs. In: *ISWC (Posters/Demos/Industry)* (2023)
24. Noy, N., Gao, Y., Jain, A., Narayanan, A., Patterson, A., Taylor, J.: Industry-scale knowledge graphs: Lessons and challenges: Five diverse technology companies show how it’s done. *Queue* **17**(2), 48–75 (2019)
25. Polleres, A., Hogan, A., Delbru, R., Umbrich, J.: RDFS and OWL reasoning for linked data. In: *Reasoning Web International Summer School*, pp. 91–149. Springer (2013)
26. Proctor, T., Young, K., Baczewski, A.D., Blume-Kohout, R.: Benchmarking quantum computers. *Nature Reviews Physics* pp. 1–14 (2025)
27. Psaraki, M., Tzitzikas, Y.: CPOI: A compact method to archive versioned RDF triple-sets. *arXiv preprint arXiv:1902.04129* (2019)
28. R. Gazzotti, F. Michel, F.G.: Cord-19 named entities knowledge graph (cord19-nekg) (2020)

29. Rajak, A., Suzuki, S., Dutta, A., Chakrabarti, B.K.: Quantum annealing: An overview. *Philosophical Transactions of the Royal Society A* **381**(2241), 20210417 (2023)
30. Sapidis, I., Zervos, V., Mountantonakis, M., Tzitzikas, Y.: Interactive and provenance-aware search and QA over documents using LLMs, RAG and knowledge graph verbalization. In: *Proc of the 29th International Conference on Theory and Practice of Digital Libraries (TPDL 2025)* (2025)
31. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* **41**(2), 303–332 (1999)
32. Tzitzikas, Y., Marketakis, Y., Minadakis, N., Mountantonakis, M., Candela, L., Mangiacrapa, F., et al.: Methods and tools for supporting the integration of stocks and fisheries. In: *Chapter in Information and Communication Technologies in Modern Agricultural Development*, Springer, 2019. Springer (2019)
33. Tzitzikas, Y., Theoharis, Y., Andreou, D.: On storage policies for semantic web repositories that support versioning. In: *European Semantic Web Conference*. pp. 705–719. Springer (2008)
34. Vrandečić, D., Krötzsch, M.: Wikidata: a free collaborative knowledgebase. *Communications of the ACM* **57**(10), 78–85 (2014)
35. Wang, K., Shen, Z., Huang, C., Wu, C.H., Dong, Y., Kanakia, A.: Microsoft academic graph: When experts are not enough. *Quantitative Science Studies* **1**(1), 396–413 (2020)
36. Wishart, D.S., Feunang, Y.D., Guo, A.C., Lo, E.J., Marcu, A., Grant, J.R., Sajed, T., Johnson, D., Li, C., Sayeeda, Z., et al.: DrugBank 5.0: a major update to the DrugBank database for 2018. *Nucleic acids research* **46**(D1), D1074–D1082 (2018)
37. Younes, A., Miller, J.: Automated method for building CNOT based quantum circuits for boolean functions. *arXiv preprint quant-ph/0304099* (2003)